

Fuzzing for Web Browser Under-Invalidation Bugs

Anonymous Submission

Abstract—Fuzzing for web browsers has traditionally focused on crash testing, such as for finding security-relevant bugs in parsers and media processing engines, and regression testing, such as for finding newly-introduced bugs in rendering code. However, fuzzing has not traditionally been used for finding correctness bugs in the implementation of the web platform standards, because writing effective test oracles for these standards is challenging. We introduce a class of correctness bugs that afflict web browser layout engines: under-invalidation bugs, which cause browsers to incompletely update internal state in response to page updates.

To combat these bugs, we introduce LAYOUTQUICKCHECK, a fuzzer for finding, minimizing, and clustering under-invalidation bugs. LAYOUTQUICKCHECK uses the browser as its own oracle, comparing an incremental and from-scratch layout of the same web page and flagging differences as under-invalidation bugs. The fuzzer generates separate initial and modified CSS states, allowing it to test whether a DOM or CSS update leaves stale layout geometry after the browser recomputes the layout. Unlike cross-browser or cross-version comparison, this self-comparison does not require a second implementation to determine the correct result.

In our evaluation, LAYOUTQUICKCHECK is able to test hundreds of fuzzer candidates per minute in both Chrome and Firefox. In both browsers, it finds dozens of bug candidates (79 in Chrome and 125 in Firefox), with LAYOUTQUICKCHECK’s custom weights further increasing the bug-discovery rate. Furthermore, across grouped Firefox reports, bug minimization reduces the number of style elements by 62.60%, while clustering reduces it by 99.74% relative to original reports and 99.32% relative to minimized reports. Despite this, these features require only 14.16% percent of the runtime. Finally, we show that LAYOUTQUICKCHECK can reproduce 7 human-submitted bugs mined from the Mozilla bug tracker and find 16 novel bugs that were reported to and accepted by developers, 7 of which were fixed by developers.

I. INTRODUCTION

Web browsers are central to modern computing, with browser-based usage corresponding to over 80% of daily work in some organizations [1]. Browser makers therefore invest heavily in fuzzing, audits, security research, standardization, shared test suites, and interoperability efforts aimed at ensuring browser correctness [2], [3], [4], [5], [6], [7].

Developers increasingly use *fuzzing*, an automated testing technique that exercises software with large numbers of randomly generated or mutated inputs, to identify and fix bugs. Browser fuzzers have been successful at finding crashes, security vulnerabilities, and rendering regressions. R2Z2 detects rendering regressions by fuzzing the output of different browser versions [2]; Metamong detects render-update bugs through fuzzing [3]; and FuzzOrigin targets UXSS vulnerabilities through origin fuzzing [4]. However, one important browser mechanism remains difficult to test systematically:

layout invalidation, which determines which parts of a page must be recomputed after changes to the DOM or CSS.

Layout invalidation is difficult because browsers attempt to reuse as much previous work as possible while still ensuring that the final page state is correct. The browser rendering pipeline includes parsing, style computation, layout, painting, and related stages [8]. After a DOM or CSS update, invalidation decides which work can be reused and which work must be recomputed. Public browser-engine trackers contain invalidation failures in Chromium, Firefox, and WebKit-based browsers [5]–[10]. Modern layout engines, including Chrome’s LayoutNG, are designed in part to reduce these correctness bugs [11], and some optimizations have historically stalled because ensuring correct invalidation was too difficult [12].

This paper introduces LAYOUTQUICKCHECK, a fuzzer for detecting under-invalidation bugs in modern web browser engines. LAYOUTQUICKCHECK systematically exercises incremental layout behavior by generating DOM and CSS updates that should force the browser to recompute affected rendering state. Its central strategy is self-comparison: the system compares the browser’s incremental layout result against its own from-scratch layout result for the same final page. This allows LAYOUTQUICKCHECK to detect stale layout state without relying on a separate browser implementation.

This paper makes the following contributions:

- We describe the challenges of extending fuzzing to browser layout invalidation and explain why existing browser fuzzers do not systematically target under-invalidation bugs.
- We design LAYOUTQUICKCHECK, a targeted fuzzer that generates DOM and CSS updates, compares incremental and fresh layout results, and reports stale layout or rendering state.
- We evaluate LAYOUTQUICKCHECK through experiments and ablation studies across XXX browser engines, measuring bug discovery, minimization, and clustering effectiveness.
- We release LAYOUTQUICKCHECK together with evaluation artifacts, generated test cases, and benchmarks at the following URL: XXX.

II. BACKGROUND, RELATED WORK, AND MOTIVATION

This section introduces the fundamental topics related to LAYOUTQUICKCHECK: web browsers, under-invalidation bugs, and the challenges of fuzzing browser layout engines.

A. Web Browsers

Web browsers are among the most widely used and economically important software systems. Their engines parse web content, compute style and layout, execute scripts, and paint pixels to the screen. Common examples include Chromium’s Blink engine [13], Mozilla’s Gecko engine [14], and Apple’s WebKit engine [15]. Browser engines require large sustained investment; for example, Open Web Advocacy estimates that Google provides roughly 90% of Chromium development funding and invests around \$1 billion annually in the web platform [16].

Unlike many traditional fuzzing targets, web browsers process highly structured inputs made up of HTML elements, CSS style rules, and JavaScript updates. These inputs can exercise DOM parsing, selector matching, style resolution, layout, and invalidation [17], [18]. Effective browser fuzzing therefore requires more than generating valid syntax: the generated inputs must be optimized toward the specific class of bugs being tested [19]–[21].

After a page has loaded, browsers rarely recompute the entire page for every change. Instead, they use invalidation logic to decide which parts of the rendering state are affected by a DOM or CSS update. Correct invalidation preserves the performance benefits of incremental rendering while ensuring that the final pixels match the page’s current state [17], [22]. Over-invalidation recomputes too much and hurts performance, while under-invalidation recomputes too little and leaves stale layout or rendering state visible [11], [18], [22].

B. Why Fuzzers Fail to Find Under-Invalidation Bugs

Most browser fuzzers are optimized for bug classes with explicit failure signals. JavaScript-engine fuzzers such as CodeAlchemist and DIE generate complex programs to expose crashes and security vulnerabilities [23], [24]. Other browser fuzzers target specialized attack surfaces such as WebGL or WebAssembly, where crashes, memory corruption, or API errors provide concrete oracles [25], [26]. These approaches are valuable, but they do not directly test whether a layout engine correctly updates stale rendering state after a DOM or CSS change.

Rendering-engine fuzzers begin to address this issue, but their goals still differ from detecting under-invalidation. RT-Fuzz mutates render-tree state but primarily evaluates robustness defects such as crashes [27]. Differential rendering fuzzers such as R2Z2, Metamong, and JANUS compare outputs across versions, executions, or browsers [2], [3], [28]. These oracles are useful, but they do not fully solve layout under-invalidation because cross-version or cross-browser differences still require determining which result is correct. Layout under-invalidation therefore needs a more targeted oracle.

C. Why it is Difficult to Isolate Under-Invalidation Bugs?

Finding an under-invalidation symptom is only the first step. Isolating the underlying bug is difficult because incremental layout is a cache-like optimization: the browser attempts to reuse previous layout results and recompute only the parts

affected by a change [29]. A stale layout result can depend on a specific DOM tree, style set, mutation, and internal invalidation decision. If an irrelevant-looking part of the generated test case is removed too early, the browser may take a different invalidation path and the bug may disappear.

The complexity of layout engines also makes bug reports hard to reduce to their root cause. Prior work observes that correct incremental layout requires reasoning about when recomputation can safely be skipped [30]. As a result, many generated tests can expose the same underlying invalidation mistake through different combinations of elements, CSS properties, and mutations.

These properties make both minimization and deduplication necessary. A useful bug report should remove unrelated elements, styles, and mutations while preserving the incremental-vs-fresh mismatch [31]. For HTML/CSS tests, reduction also needs to respect document structure so minimized tests remain valid and meaningful [32], [33].

Motivation: the need for an under-invalidation browser layout fuzzer. Existing browser fuzzers struggle to detect layout under-invalidation bugs. LAYOUTQUICKCHECK addresses this gap by making browser layout invalidation testable with a targeted oracle, minimization, and deduplication.

III. CHALLENGES AND SOLUTIONS

To address the limitations of conventional browser fuzzing for layout invalidation correctness, we introduce LAYOUTQUICKCHECK: a system for finding, minimizing, and grouping under-invalidation bugs in modern web browser layout engines. In the following, we outline core challenges that shape LAYOUTQUICKCHECK’s design, along with their solutions aimed at effectively fuzzing browser layout invalidation.

A. Challenge 1: Defining a Correctness Oracle

A central challenge in testing layout invalidation is that there is no obvious external ground truth for the correct layout of a generated page. Cross-browser comparisons can expose rendering differences, but those differences do not necessarily identify an invalidation bug because browsers may make different implementation, feature-support, or rendering choices. Crash-based or assertion-based fuzzing is also insufficient because an under-invalidation bug can produce a visually incorrect layout while the browser continues running normally. The difficult part is therefore not only generating a page that changes layout, but also deciding whether the browser reused stale layout information after that change.

No external ground truth: A generated HTML and CSS test case does not come with a known correct layout. Comparing against another browser can identify that two implementations disagree, but it cannot determine which browser is wrong or whether the difference is caused by invalidation. LAYOUTQUICKCHECK therefore needs an oracle that does not depend on a separate browser or reference implementation.

Non-crashing failures: Under-invalidation bugs often do not crash the browser or violate an explicit assertion. Instead, the browser may continue executing while displaying stale geometry. This makes conventional crash-based fuzzing insufficient, because the fuzzer needs to detect a semantic layout mismatch rather than a process failure.

Path-dependent correctness: Under-invalidation depends on an execution path, not just a final page state. The same HTML and CSS may be correct when the browser computes layout from scratch, but incorrect when the browser reaches that state through an incremental update. A useful oracle must therefore compare two ways of reaching the same final document state: one through the normal dynamic update path and one through a fresh layout computation.

Solution 1: LAYOUTQUICKCHECK addresses this challenge using a self-differential testing oracle. For each generated test case, LAYOUTQUICKCHECK compares the browser’s normal incremental layout after generated style changes against a from-scratch layout to identify a bug.

B. Challenge 2: Triggering Under-Invalidation Behavior

Valid HTML and CSS are not necessarily useful for testing layout invalidation. Under-invalidation occurs only when a page first establishes a layout state, then receives a meaningful DOM or CSS update, and the browser fails to recompute geometry affected by that update. An unconstrained fuzzer can produce many accepted pages without exercising these incremental-layout paths. LAYOUTQUICKCHECK therefore biases generation toward CSS properties and mutations that are likely to change layout, so that each test is more likely to trigger the browser’s invalidation logic.

Display: The `display` property is a useful target because it determines the layout mode used for an element and changes how its children participate in layout. Changing `display` can move an element between block, inline, flex, grid, table, or hidden states, each of which requires different layout behavior. These changes can affect both the modified element and its descendants, they provide strong opportunities to expose missed invalidation decisions in incremental layout engines [11], [29].

Grid: Grid layout creates many invalidation opportunities because it introduces relationships between a container and its children. Changing grid templates, gaps, placement rules, or auto-flow can reposition multiple children from a single style update. This makes grid a strong target for LAYOUTQUICKCHECK because a missed invalidation can leave incorrect child positions or incorrect container geometry. Such dependent geometry is precisely the kind of cached result that incremental-layout implementations must invalidate correctly [29], [30].

Flow-root: The `display:flow-root` value is important because it establishes a new block formatting context and changes the boundary at which surrounding layout is resolved. Mutating it can therefore change both the target box and the geometry of neighboring content, making it useful for exercising invalidation propagation [29].

Width and height: Width and height changes directly affect an element’s box geometry. When an element changes size, the

browser may need to recompute that element, its descendants, siblings, and sometimes ancestors. These mutations are useful because layout engines often appears as an obvious mismatch in element size or position.

Percentage and pixel sizes: Percentage sizes test parent-dependent layout behavior because the final size of an element depends on its containing block. Pixel sizes create more direct change to the geometry possibly causing conflict issues with surrounding elements. Using both lets LAYOUTQUICKCHECK exercise dependent layout constraints to produce failures that are easy to observe and reproduce. General browser fuzzers need not preserve such a parent-dependent update relationship, because their target oracles are typically crashes, security violations, or externally observed output differences [2], [4], [23], [24].

Transforms: Transform changes affect visual position and reported geometry. They provide a controlled update whose effect can be checked through the geometry oracle while interacting with containing blocks and descendant coordinates. This makes them useful for distinguishing a stale incremental result from a freshly computed final layout.

Noise reduction: LAYOUTQUICKCHECK disables properties such as `writing-mode`, `content-visibility`, `padding`, and `table display` values because they are less likely to produce under-invalidation bugs. Instead, these properties often add unnecessary complexity to the generated tests. Removing them keeps the generated tests focused on mutations that are more likely to expose missed invalidation decisions and helps produce smaller, clearer bug reports. This focus is important because unconstrained random inputs can make an observed rendering difference hard to attribute to one invalidation decision [2], [3], [28].

Solution 2: LAYOUTQUICKCHECK includes a generation strategy for steering generated inputs toward page updates that are likely to trigger layout under-invalidation bugs. The system focuses on CSS properties and mutations that should force layout recomputation, increasing the chance that a missed invalidation decision becomes observable.

C. Challenge 3: Reducing Report Complexity

Generated pages can contain many unrelated elements, declarations, and mutations. Likely only a small portion of the page causes a bug thus the original report may obscure the relevant DOM structure and CSS transition. The output can also contain many reports that are syntactically different but match a previously observed pattern. Without reduction and grouping, developers must repeatedly inspect large, redundant reports. At the same time fuzzers commonly generate many HTML and CSS variations that exercise the same underlying defect, so treating every report as a new bug would overstate the diversity of the findings. However, merging reports too aggressively could hide genuinely different bugs.

Preserving the bug trigger: Minimization is difficult because under-invalidation bugs depend on a precise sequence of layout states. The browser first lays out the original page, then applies a DOM or CSS update, and then decides which cached

layout information can be reused. If the minimizer removes an element, style, or mutation that seems unrelated but changes this invalidation path, the bug may disappear even though the original conditions were present.

Separating base and modified styles: To identify which declarations are essential to the transition, LAYOUTQUICKCHECK moves one modified style declaration at a time into the base style map. If the mismatch persists after the move, that declaration was not required as an incremental update and can be removed from the modified state. If the mismatch disappears, the declaration remains modified because changing it after the initial layout is part of the bug trigger.

Avoiding redundancy: In addition full minimization requires many browser executions. Before minimizing a new report, LAYOUTQUICKCHECK can compare it with existing structural group rules. If it matches a group, then the system can assume that there is a common pattern and can retain the shared rule and minimized representative rather than fully minimizing an additional variation. Unmatched reports are minimized and can establish new groups.

Identifying repeated patterns: To determine whether reports represent recurring behavior or potentially new bugs, LAYOUTQUICKCHECK records the DOM structure, base styles, modified styles, and affected element relationships. It uses this representation to group reports that match an existing structural rule, while preserving unmatched reports as separate candidates. The resulting groups are evidence of recurring patterns, not proof of a shared root cause; similarly, unmatched reports are candidates for separate validation rather than confirmed bugs.

Solution 3: LAYOUTQUICKCHECK minimizes reports while preserving the state transition that exposes the bug, then groups structurally equivalent reports around a shared representative. This reduces the HTML and CSS a developer must inspect, avoids spending full minimization cost on reports already represented by a group, and separates repeated variants from potentially distinct candidates. The resulting reports are then suitable for browser-developer review, which provides the final evidence that a candidate is real and novel.

IV. IMPLEMENTATION

LAYOUTQUICKCHECK is implemented in Python as an end-to-end, feedback-free fuzzing pipeline for browser layout invalidation. The implementation separates a test case from its serialization, executes the serialized page in a real browser through Selenium WebDriver. A under-invalidation failure is identified when there is a discrepancy between incremental and fresh layout computations. The same pipeline then classifies, minimizes, and sorts the result. At a high level, a run has six stages: (1) configuration and browser selection, (2) generation of a DOM and two style states, (3) execution of the layout oracle, (4) minimization of a bug-triggering test case, (5) optional structural grouping and minimization skipping, and (6) report and metric production. The following sections describe these stages and the concrete artifacts exchanged between them.

A. Configuration, Execution Model, and Test Subjects

The command-line runner accepts a JSON configuration file and initializes a process-wide configuration object before any pages are generated. The configuration supplies two paths: a directory for persistent bug reports and a directory for temporary generated pages. It also supplies a list of browser variants and a map from CSS property names to generation weights. The runner supports a bug limit, test limit, and crash limit. The bug limit stops the experiment when it reaches the specified cumulative number of bug reports, while the test limit stops it at the specified cumulative number of executed test cases. The crash limit stops the current process after the specified number of browser or WebDriver failures; crashes from an earlier resumed process remain in the reported total but do not consume the new process's allowance. The `--no-sort` option disables structural classification and grouping, causing each candidate to be fully minimized and saved as an individual report rather than compared with existing bug groups. With no explicit limit, the runner remains active until it is interrupted or the crash limit is reached.

A browser variant specifies a browser type together with its WebDriver path, viewport, and optional browser settings. After reading a variant from the configuration, LAYOUTQUICKCHECK builds the corresponding Selenium WebDriver for Chrome, Firefox, or Safari. The variant marked `target` is used for the main fuzzing loop. Internally, each generated test is represented as a `RunSubject`:

- an `ElementTree`, which is a nested representation of the generated DOM;
- a base `StyleMap`, containing declarations applied in the initial document; and
- a modified `StyleMap`, containing declarations installed later by JavaScript.

Both style maps are indexed by generated element identifier and then by CSS property name. Keeping base and modified declarations separate captures the state transition being tested: the initial layout is computed from the base map, and the incremental update is the modified map.

B. DOM and CSS Generation

LAYOUTQUICKCHECK generates a body subtree recursively. The current generator uses `div` elements and text nodes. Each `div` receives a unique alphabetic identifier derived from a UUID so that it can be addressed from both CSS and the generated JavaScript. Generation is probabilistic: the body is very likely to receive children, while nested `divs` have a smaller probability of receiving one or more children. Text nodes contain one or more generated sentences, optionally separated by a newline. This produces trees with both nested boxes and text-dependent sizing behavior.

For every non-text element, the generator independently constructs base and modified style maps. It walks a repository-maintained catalogue of CSS properties and their supported value generators. The configured weight is first constrained to the range 0–100 and then divided by 100 to obtain the

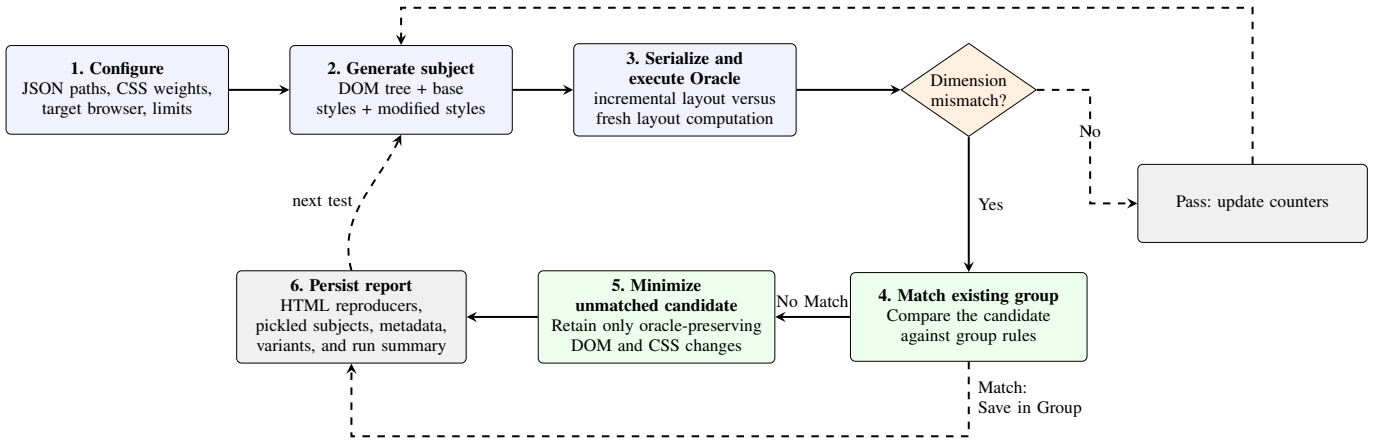


Fig. 1: LAYOUTQUICKCHECK pipeline. Each generated subject is tested by a browser-resident layout oracle. Passing subjects advance the fuzzing loop; candidates are structurally classified, minimized when necessary, and persisted with their reproduction artifacts and cumulative metrics.

probability of selecting that property, i.e., $P(\text{select}) = w/100$. Thus, a weight of zero disables the property, a weight of 40 selects it with probability 0.40, and a weight of 100 selects it whenever it is considered. This random decision is made independently for each non-text element and for each of its base and modified style maps. If selected, a property-specific generator produces a valid length or keyword value. Unspecified properties use a default selection weight, allowing an experiment to focus on selected properties without having to enumerate the entire catalogue.

This design makes property weighting an input-distribution control rather than a change to the oracle. A configuration can emphasize properties that are relevant to a particular layout behavior while reducing or disabling the selection of less relevant properties. The weighted and unweighted experiments in Section V therefore execute the same generation and oracle pipeline but sample different style distributions.

C. Page Serialization and the Layout Oracle

Each `RunSubject` is serialized into a self-contained temporary HTML file. The serializer recursively emits the element tree, places base declarations in each element’s inline `style` attribute, and emits a JavaScript `makeStyleChanges()` function. That function assigns every declaration in the modified style map through the element’s `style` object. The generated page also includes a small JavaScript support file that implements the geometry collection and comparison operations. The runner uses a temporary HTML file to execute each generated test case.

The oracle compares two executions that should have final markup and style declarations an incremental and a fresh layout computation:

- 1) Selenium navigates the target browser to the generated local file URL and waits for the document body to be present.

- 2) `makeStyleChanges()` applies the modified declarations to the live document. This is the incremental path whose invalidation behavior is under test.
- 3) The page records `getBoundingClientRect()` for every DOM element. Each observation includes the element identifier, tag name, and its dimensions.
- 4) The page assigns the document element’s `innerHTML` property to itself. This reconstruct forces the the browser to establish a fresh layout for that final state again.
- 5) The page collects the dimensions again and compares the two sets. A difference in `x`, `y`, `left`, `right`, `top`, `bottom`, `width`, or `height` for the same element is returned to Python.

The browser-side comparison returns a list of differing elements. Each entry identifies the affected element and records the attributes that differ as well as the values after incremental application and after document reconstruction. An empty list is a pass; a nonempty list is represented as a layout-bug result. WebDriver failures produce a separate page-crash result, while a page-load timeout is reported by the harness. Recording the individual geometric fields is useful during triage because it distinguishes, for example, a stale width from a descendant-position mismatch.

The oracle detects a candidate failure when the incremental and reconstructed layout paths produce different element dimensions in the selected browser and viewport.

D. Fuzzing Loop and Safe Repository

The runner first constructs a small repository of known-safe subjects. It generates and executes subjects until the configured safe directory contains 25 passing serialized `RunSubject` objects. The configured report directory is a persistent, cumulative bug repository: it contains all saved reports, groups, safe samples, and the run summary for that configuration. When a later invocation uses the same directory, it continues the same repository and aggregates newly discovered reports with the existing ones rather than creating a separate collection. These

safe samples are used to prevent the grouping algorithm from accepting a rule that would also match known non-bug inputs.

The main loop then generates one subject, executes the oracle, increments the pass counter for a clean result, and enters post-processing only for a candidate. After every completed test, the runner updates the run summary. The summary records executed tests, passes, candidate bugs, failures that no longer reproduce during reduction, reports with no remaining modified style, crashes, elapsed time, and cumulative minimization and sorting time. It also counts standalone reports, grouped reports, and groups by scanning the report directory. [**TODO: Not in final version**] Threshold snapshots are written at every ten candidates and every 60 seconds of accumulated runtime, which supports time-series plots.

The runner resumes these cumulative statistics if a report directory already contains `run_summary.json`. Resuming with the same directory retains the existing bug reports, bug groups, grouping rules, and safe samples. New reports are therefore compared with the accumulated repository and can join an existing group instead of forcing the grouping process to start again. This makes interrupted long-running experiments continue as one statistical run, without losing their prior discovery, grouping, or measurement state.

E. Delta Debugging

Candidate pages are usually much larger than the condition required to reproduce a mismatch. `LAYOUTQUICKCHECK` applies delta debugging to reduce a bug-triggering test case while preserving the oracle result. For each proposed simplification, it deep-copies the current subject, executes the candidate in the target browser, and retains the copy only if the result remains a bug. Every accepted reduction becomes the new subject, so later reductions are tested against the smallest reproducer found so far. The delta-debugging process uses the following proposal families in order:

- 1) remove each element, including its associated base and modified style entries;
- 2) remove every style map for one element at a time;
- 3) remove individual base or modified declarations;
- 4) move one modified declaration into the base map, thereby testing whether that declaration must be an incremental change rather than merely being present in the final state;
- 5) replace generated length values with `20px` or `-20px`, preserving only their sign; and
- 6) apply readability-oriented transformations, including minimum box sizes, background colors, short identifiers, and an optional in-page control overlay.

The fourth step, implemented by `Minify_MoveStyleChangesToFirstLoad`, is particularly important for this bug class. If moving a declaration to the initial state preserves the mismatch, it is not necessary for the incremental transition and can be removed from the set of modified properties. Conversely, declarations that cannot be moved identify the state change that is more likely to matter to invalidation. The final oracle

run is recorded again after all proposal families have been exhausted. If the candidate no longer reproduces, it is counted as a false positive rather than saved as a confirmed layout report. A layout mismatch with an empty modified-style map is also counted separately because it no longer expresses the intended incremental CSS-change scenario.

F. Structural Grouping and Minimization Skipping

Full reduction is expensive when many generated subjects trigger a pattern already represented in the report directory. With sorting enabled, `LAYOUTQUICKCHECK` therefore attempts to match a new candidate against existing bug groups before full minimization. A subject is converted to a tree in which each node contains its tag, identifier, base styles, modified styles, and children; text is represented explicitly. The first node with modified styles is used as the start node for a rule because it anchors comparison on the update of interest. The rule represents the DOM structure relative to this anchor: it compares the modified node and its ancestor/descendant structure, rather than depending on that node's absolute location in the full page tree. This allows structurally similar update patterns to match even when they occur in different generated pages.

A group stores a merged representative tree and an extracted JSON rule. To form a candidate rule, the implementation merges two trees recursively. Equal tags, values, and declarations are retained. Disagreements are represented by the wildcard value `diff`. The resulting rule consists of an ordered HTML pattern and the common base and modified style requirements at the start node. Matching checks both components: the incoming subject must contain the ordered structural pattern and at least one matching element must have the required base and modified declarations. Wildcards permit variation where the merged reports disagree while retaining the shared structure and update signal.

Before accepting either a match or a new group, the rule is validated against the safe repository. Rules with no modified-style requirement or with a trivial single-element pattern are rejected, and a rule is rejected if it matches any safe sample. This is a conservative guard against a broad structural description that would classify ordinary pages as bug instances. It does not prove semantic equivalence, but it makes group promotion depend on both bug examples and known non-bug examples.

If an incoming candidate matches an existing rule, the runner skips the full reduction and saves it as another instance under that group. Otherwise, it minimizes the candidate first. The final minimized subject is then compared against existing groups and against standalone reports. Matching a group adds the new instance to that group. Matching a standalone report promotes the two reports into a newly created `bug-group-*` directory and writes the extracted rule. After every group update, group artifacts are recomputed so the merged representative and rule correspond to the instances currently stored in that directory. The sorting time is measured separately and added to the run summary alongside minimization time;

this makes the cost and benefit of grouping measurable in Section V.

G. Report Layout, Reproduction, and Variant Results

Each saved report is placed in either a standalone `bug-*` directory or a child directory of a `bug-group-*` directory. To keep the bug repository lightweight, the report stores pickled forms of the pre-reduction and minimized `RunSubject`, rather than retaining separate HTML versions of both pages. These structured artifacts preserve the DOM and style states needed to rerun reduction or recompute grouping rules.

The accompanying `data.json` records the save timestamp, bug type, all styles used, base and modified style names, the geometric differences returned by the oracle, whether minimization was skipped, the matched rule, and the time spent sorting and minimizing. After minimization, the runner executes each configured browser variant and records browser name, browser version, operating-system details, viewport, variant description, and whether that variant also detects the bug. These fields distinguish the target run from supplementary checks and make reports usable when browser versions or command-line flags change.

Together, these artifacts allow a report to be inspected at several levels: the original input, the minimized browser reproducer, the exact geometry disagreement, the structured subject used by automated tooling, and its relationship to a group. This is the implementation basis for the throughput, bug-rate, minimization-time, and grouping measurements reported in the evaluation.

V. EVALUATION

Our evaluation of LAYOUTQUICKCHECK is guided by these fundamental questions:

RQ1: Is LAYOUTQUICKCHECK at triggering an under-invalidation bugs?

RQ2: Do minimization and clustering reduce report complexity?

RQ3: Does LAYOUTQUICKCHECK find real, novel under-invalidation bugs?

Benchmarks: Table I summarizes the browser targets evaluated by LAYOUTQUICKCHECK. Our evaluation uses Chromium, which exercises the Blink layout engine, and Firefox, which exercises the Gecko layout engine. LAYOUTQUICKCHECK uses ChromeDriver and GeckoDriver, respectively.

Browser	Rendering Engine	Driver
Chrome	Blink	ChromeDriver
Firefox	Gecko	GeckoDriver

TABLE I: Our browser-based layout fuzzing evaluation benchmarks.

Experiment Setup: Because no existing fuzzer specifically targets under-invalidation bugs, our evaluation focuses on LAYOUTQUICKCHECK’s efficiency and quality of the results it produces. For RQ1, we run each browser with and without property weights for 60 minutes. We save several

characteristics of these runs including speed(number of tests ran per minute), bugs found, rate of bug discovery, and the overall execution time. During the 60 minute window every minute a snapshot of these metrics were saved to accurately track the metrics across time. This data is represented in ?? which shows the bugs detected over execution time and II which showcases metrics at the end of the 60 minute window. For RQ2, the evaluations compares Firefox runs with sorting enabled and disabled over a nine hour execution. We measure the number of css properties in the original reports, minimized reports and clustered reports, cumulative post-processing time of minimization and clustering, and execution time. Measuring total css properties takes into account only the bugs that are within a bug group as to not inflate the total css properties of grouped or minimized bugs that were never grouped. For RQ3, we report the status of novel bugs submitted to browser developers and use report groups and single bugs to describe the diversity of the report repository.

A. RQ1: Finding Under-Invalidation Reports

RQ1 establishes whether LAYOUTQUICKCHECK can repeatedly drive independent browser engines into under-invalidation behavior. Property weighting is a secondary comparison: it biases generation toward selected CSS properties, while the unweighted configuration samples from the same property space without that bias. Figure 2 shows that reports continue to accumulate during the 60-minute window, and Table II distinguishes report count from throughput and report-triggering rate.

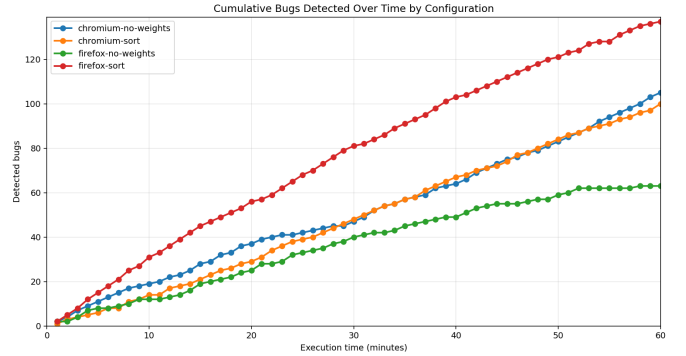


Fig. 2: Cumulative bug reports for the weighted and no-weight configurations over the 60-minute evaluation window.

Browser	No weights			Weights		
	Speed	Bugs	Rate	Speed	Bugs	Rate
Chromium	196.8	105	0.889%	558.0	100	0.299%
Firefox	495.9	63	0.212%	492.8	137	0.463%

TABLE II: RQ1 bug-discovery results across browser and property-weight configurations over a 60-minute execution window. Speed is measured in executed tests per minute; rate is bugs divided by executed tests.

Outcomes: Every configuration finds dozens of reports within 60 minutes: Chromium finds 105 reports without property weights and 100 reports with property weights, while Fire-

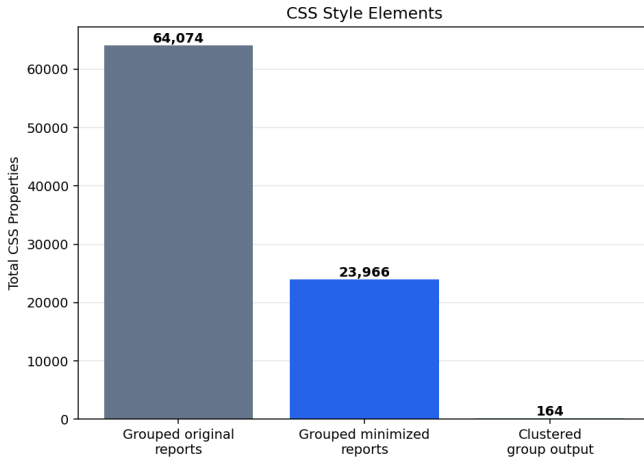


Fig. 3: Total CSS style elements in grouped Firefox reports before minimization, after minimization, and after clustering.

fox finds 63 and 137 reports, respectively. The nonzero rates in both Blink and Gecko show that LAYOUTQUICKCHECK reaches the target behavior in two independent layout engines, rather than depending on one browser-specific implementation.

The weighted comparison explains how configuration choice affects that ability. In Firefox, weighting raises the report count from 63 to 137 and the report-triggering rate from 0.212% to 0.463%, while throughput remains nearly unchanged (495.9 versus 492.8 tests per minute). In Chromium, weighting increases throughput but does not improve discovery rate: the unweighted configuration produces more reports (105 versus 100) and a higher report-triggering rate (0.889% versus 0.299%). Thus, higher throughput alone does not explain the usefulness of property weights depends on the browser engine and the properties they emphasize.

RQ1: LAYOUTQUICKCHECK finds under-invalidation reports in both Chromium and Firefox within one hour. Property weights improves discovery rate, but their benefit is browser-dependent rather than universal.

B. RQ2: Reducing Developer Triage Work

Finding a bug is most useful when it is easy to understand and analyze. RQ2 asks whether minimization and clustering reduce that burden, in addition it is important to examine the efficiency impacts. Minimization removes unnecessary CSS style elements from a report. Clustering recognizes repeated reports and retains a shared rule and minimized representative, which can in turn be used to avoid full minimization for every bug found. We therefore evaluate two linked outcomes: how much smaller the developer-facing reports become, and whether the extra processing reduces fuzzing throughput. Figure 3 measures the former; Figure 4 measures the latter.

Outcomes: The report-size results show that the clustering and minimization pipeline meaningfully changes the volume of a bug analysis needed. Minimization reduces the style elements

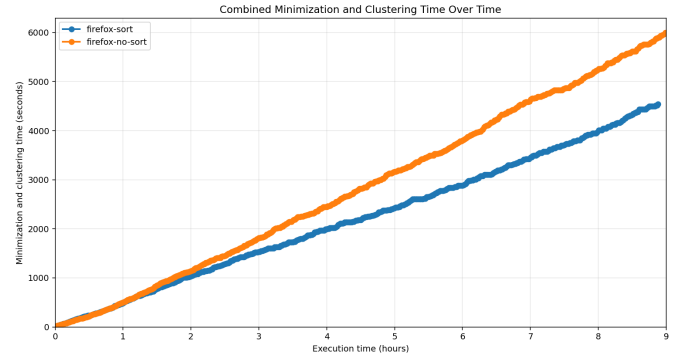


Fig. 4: Cumulative minimization and clustering time for Firefox sorting and no-sorting configurations.

in grouped reports by 62.60% relative to the original reports. Combining minimization with clustering reduces the style-element volume by 99.74% relative to originals and by 99.32% relative to minimized reports. Clustering removes a significant amount of repeated work across reports that represent the same behavior. LAYOUTQUICKCHECK retains the group’s shared rule and minimized representative instead of independently minimizing and inspecting the new report. Developers can therefore focus on one representative report per behavior rather than repeatedly examining equivalent reports.

In addition the minimization-and-sorting pipeline accounts for only 14.16% of the Firefox sorting run’s runtime. Moreover, specifically the sorting of bug reports into bug groups does not introduce an overhead; instead, it reduces post-processing cost when compared to simply minimizing each bug. At the latest common snapshot, the sorting configuration executes $\sim 130,640$ test cases, compared with $\sim 126,210$ without sorting, while spending $\sim 4,550$ seconds on minimization and sorting rather than $\sim 5,890$ seconds without sorting. When a report matches an existing group, LAYOUTQUICKCHECK can reuse that group’s representative instead of fully minimizing the new report. Consequently, sorting reduces cumulative post-processing time by $\sim 22.6\%$ and enables $\sim 4,430$ additional tests to run.

RQ2: Minimization and clustering make reports substantially easier to dissect while preserving, and in this experiment improving, fuzzing throughput. They reduce both the size of individual reports and the repeated work spent on reports already represented by a bug group.

C. RQ3: Finding Real, Novel Bugs

The candidate reports measured in RQ1 are not, by themselves, evidence of distinct root-cause bugs. RQ3 therefore asks whether LAYOUTQUICKCHECK produces reports that survive external developer review. We submitted 16 novel under-invalidation bugs discovered by LAYOUTQUICKCHECK to Chrome and Firefox developers; Table III records their current statuses. 7 of these reports were fixed or closed by developers. We also reproduce 7 human-submitted Mozilla bugs,

Browser	Bug ID	Brief description	Status
Chrome	1137427	Repeated white-space application failure	Fixed
Chrome	1166887	Dynamic table-caption vertical order swap	Fixed
Chrome	1189755	Fixed-position grid layout under-invalidation	Fixed
Chrome	1189762	None-to-inline-grid display invalidation	Open
Chrome	1190220	Grid layout page crash	Fixed
Chrome	1219641	Inline-grid list-item text-indent invalidation	Open
Chrome	1224721	Grid layout under-invalidation report	Fixed
Chrome	1229681	Grid child display-none invalidation	Open
Firefox	1721719	Table row-group height inconsistency	Open
Firefox	1724982	Table min-height overflow discrepancy	Open
Firefox	1724991	Dynamic break-spaces invalidation inconsistency	Open
Firefox	1733276	Display-inline margin line-wrapping inconsistency	Open
Firefox	1735376	Grid table-row height inconsistency	Closed
Firefox	1735931	Percent-sized grid-item height inconsistency	Closed
Firefox	1748891	Sticky padding scrollport position inconsistency	Open
Firefox	1724999	Layout Quick Check metabug	Open

TABLE III: Status of bug reports submitted to Chrome and Firefox.

showing that the oracle can reach known under-invalidation behavior.

Clustering provides complementary evidence about report diversity before manual validation. Figure 6 shows how reports are divided into bug groups and single bugs over time, while Figure 5 shows the final size distribution of the Firefox groups. **Outcomes:** Table III establishes that LAYOUTQUICKCHECK’s reports are actionable beyond the experimental oracle. It produced 16 novel reports submitted to browser developers, and 7 were fixed or closed. These outcomes provide external evidence that the tool reaches real under-invalidation defects rather than only reporting test artifacts.

The clustering results provide evidence that LAYOUTQUICKCHECK discovers a diverse set of potential bugs rather than merely generating many syntactic variations of the same behavior. The clustering process groups reports with shared behavior while retaining a minimized representative for each group. At the latest common snapshot, the Firefox sorting run produces 564 reports across 45 groups and 38 unmatched reports. The number of groups, each represented by distinct HTML and CSS combinations, indicates that the report repository contains substantial behavioral variation rather than a single bug expressed repeatedly. These counts are evidence of potential diversity, not confirmed root-cause-bug counts; confirmation of novelty comes from the submitted reports reviewed by browser developers in Table III.

RQ3: LAYOUTQUICKCHECK finds real, novel under-invalidation bugs: it produced 16 reports submitted to browser developers, of which 7 were fixed or closed. Clustering helps further separate recurring reports from candidates to make independent validation easier.

VI. DISCUSSION & THREATS TO VALIDITY

Optimizing Runtime Throughput:

Several parts of the workflow can limit throughput, particularly test generation, minimization, and sorting. More efficient minimization and sorting algorithms could reduce this overhead. For example, minimization could use a divide-and-conquer strategy that first attempts to remove groups of

HTML nodes or CSS declarations, then progressively tests smaller groups when necessary. Unlike the current element-by-element and declaration-by-declaration approach, this could reduce the number of browser executions required to find a compact reproducer.

Test generation could be improved by detecting and discarding semantically redundant mutations or combinations, such as style changes that produce the same computed value or cannot affect the generated element’s layout. This would allow more browser executions to target meaningful layout interactions.

Finally, generation, minimization, and sorting can be parallelized across multiple browser instances or machines. Candidate bugs could be queued for minimization and sorting while other workers continue generating and executing new tests. This would increase throughput, although it introduces resource-management, synchronization, and duplicate-reporting challenges.

Despite these throughput limitations, the current workflow remains effective because it successfully generates, detects, minimizes, and organizes reproducible browser-layout bugs.

Supporting Other Browsers:

Supporting a browser requires a browser-specific configuration, a compatible automation driver, and validation that the browser exposes layout information consistently enough for the test oracle. These requirements make it straightforward to support widely used browsers with robust WebDriver implementations, but they limit practical testing of uncommon browsers and older browser versions.

Less frequently tested browsers may contain defects that are not found by testing dominant browser engines. Supporting these browsers could therefore reveal bugs in engines that receive less testing and maintenance. However, support for additional browsers has diminishing returns because most web users are concentrated in a small number of major browser engines.

The supported Chromium, WebKit/Safari, and Firefox configurations cover most browser usage. Worldwide in May 2026, Chrome accounted for 70.25% of browser usage, Safari for 15.72%, and Firefox for 5.14%; together, these browsers account for 91.11% of usage [34]. This leaves only 8.89% of usage outside of the main target of LAYOUTQUICKCHECK.

Nevertheless, the current implementation remains effective because it supports major browser engines with mature automation tooling and substantial user populations.

Increasing Bug Rate:

The bug detection rate directly affects the efficiency of the tool, since a higher rate produces more candidate bugs per executed test case. However, repeatedly finding the same underlying defect is less useful than discovering distinct bugs. Nevertheless, the tool remains effective because sorting and grouping reduce duplicate reports and help distinguish recurring bug patterns.

One approach to increasing the bug rate is to identify a stronger static style-weight configuration before a testing

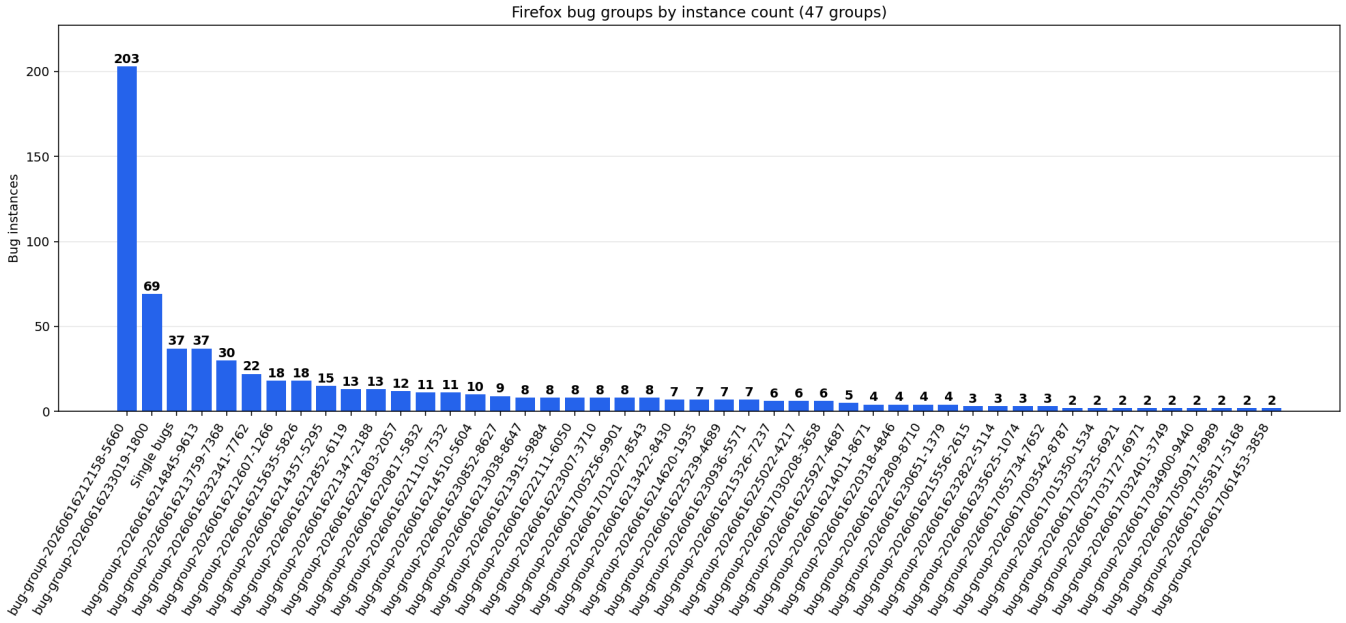


Fig. 5: Firefox bug-group sizes, with all single bugs combined into one bar.

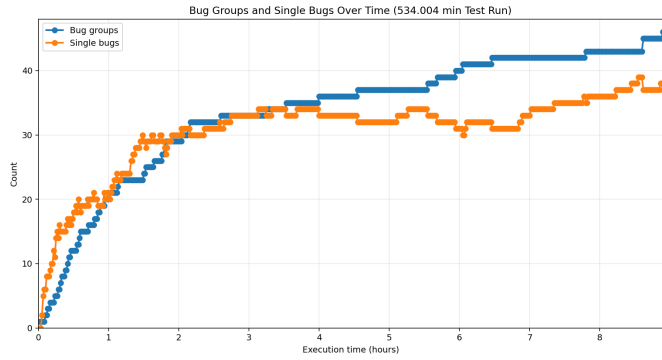


Fig. 6: Bug-group and single-bug counts by total detected reports in the Firefox sorting run.

run begins. Results from earlier runs can identify HTML structures, CSS properties, and dynamic style changes that have produced layout inconsistencies most often. These observations can then be used to assign greater initial weights to promising properties while preserving lower probabilities for less frequently selected properties.

Separately, a feedback-driven system could dynamically adjust style-selection probabilities while a testing run is running. It could increase the weights of properties associated with novel layout discrepancies and reduce the weights of configurations that repeatedly produce uninteresting or duplicate results. The system should retain a degree of random exploration so that it does not become overly focused on known bug patterns. Nevertheless, the current random generation strategy remains useful because it can discover unexpected bug classes without requiring prior knowledge of effective configurations.

VII. CONCLUSION

ACKNOWLEDGMENTS

REFERENCES

- [1] F. Montenegro, “The state of workforce security: Key insights for it and security leaders,” Palo Alto Networks, Tech. Rep., Jan. 2025.
- [2] S. Song, J. Hur, S. Kim, P. Rogers, and B. Lee, “R2Z2: Detecting rendering regressions in web browsers through differential fuzz testing,” in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE ’22. ACM, 2022.
- [3] S. Song and B. Lee, “Metamong: Detecting render-update bugs in web browsers through fuzzing,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE ’23. ACM, 2023.
- [4] S. Kim, Y. M. Kim, J. Hur, S. Song, G. Lee, and B. Lee, “FuzzOrigin: Detecting UXSS vulnerabilities in browsers through origin fuzzing,” in *Proceedings of the 31st USENIX Security Symposium*, ser. USENIX Security ’22. USENIX Association, 2022. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/kim>
- [5] Chromium, “Issue 497183437,” Chromium Issue Tracker. [Online]. Available: <https://issues.chromium.org/issues/497183437>
- [6] —, “Issue 379886422 resources,” Chromium Issue Tracker. [Online]. Available: <https://issues.chromium.org/issues/379886422/resources>
- [7] Mozilla, “Bug 1452426,” Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=1452426
- [8] —, “Bug 539356,” Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=539356
- [9] WebKit, “Bug 233421,” WebKit Bugzilla. [Online]. Available: https://bugs.webkit.org/show_bug.cgi?id=233421
- [10] —, “Bug 97801,” WebKit Bugzilla. [Online]. Available: https://www2.webkit.org/show_bug.cgi?id=97801
- [11] I. Kilpatrick and K. Ishii, “RenderingNG deep-dive: LayoutNG,” Chrome for Developers, 2021, last updated October 8, 2021. [Online]. Available: <https://developer.chrome.com/docs/chromium/layoutng?hl=en>
- [12] MozillaWiki, “Gecko: Reflow refactoring,” MozillaWiki, 2006, landed December 7, 2006. [Online]. Available: https://wiki.mozilla.org/Gecko:Reflow_Refactoring
- [13] The Chromium Projects, “Blink: Rendering engine,” The Chromium Projects, accessed June 2, 2026. [Online]. Available: <https://www.chromium.org/blink/>
- [14] Mozilla, “Gecko,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: <https://firefox-source-docs.mozilla.org/overview/gecko.html>

- [15] WebKit, “Webkit,” WebKit, accessed June 2, 2026. [Online]. Available: <https://webkit.org/>
- [16] Open Web Advocacy, “Break google’s search monopoly without breaking the web,” Open Web Advocacy, Apr. 2025, published April 23, 2025. [Online]. Available: <https://open-web-advocacy.org/blog/break-googles-search-monopoly-without-breaking-the-web/>
- [17] M. Kosaka, “Inside look at modern web browser, part 3,” Chrome for Developers, 2018. [Online]. Available: <https://developer.chrome.com/blog/inside-browser-part3>
- [18] MDN Web Docs, “Web performance,” MDN Web Docs, accessed June 2, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/Performance>
- [19] Mozilla, “Fuzzing,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: <https://firefox-source-docs.mozilla.org/tools/fuzzing/index.html>
- [20] P. Ventuzelo, “Practical web browser fuzzing,” Ringzer0 Training, 2023. [Online]. Available: <https://ringzer0.training/practical-web-browser-fuzzing/>
- [21] I. Araoz Severiche, “Web fuzzing: A practical testing methodology,” Medium, Feb. 2026, published February 7, 2026. [Online]. Available: <https://iaraoz.medium.com/web-fuzzing-a-practical-testing-methodology-1f24198e2ddc>
- [22] Mozilla, “Performance best practices for firefox front-end engineers,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: <https://firefox-source-docs.mozilla.org/performance/bestpractices.html>
- [23] H. Han, D. Oh, and S. K. Cha, “CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines,” in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS ’19, 2019.
- [24] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, “Fuzzing JavaScript engines with aspect-preserving mutation,” in *Proceedings of the IEEE Symposium on Security and Privacy*, 2020.
- [25] H. Peng, Z. Yao, A. Amiri Sani, D. J. Tian, and M. Payer, “GLeeFuzz: Fuzzing WebGL through error message guided mutation,” in *Proceedings of the USENIX Security Symposium*, 2023.
- [26] O. Draissi, T. Cloosters, D. Klein, M. Rodler, M. Musch, M. Johns, and L. Davi, “Wemby’s web: Hunting for memory corruption in WebAssembly,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.
- [27] Y. Zeng, Y. Wu, X. Lu, and C. Zhang, “RTFuzz: Fuzzing browsers via efficient render tree mutation,” *Computers & Security*, vol. 161, p. 104756, 2026.
- [28] C. Zhou, Q. Zhang, B. Qian, and Y. Jiang, “JANUS: Detecting rendering bugs in web browsers via visual delta consistency,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, ser. ICSE ’25. IEEE, 2025.
- [29] P. Panchekha and C. Harrelson, *Reusing Previous Computations*. Oxford University Press, 2024, pp. 443–494. [Online]. Available: <https://browser.engineering/invalidation.html>
- [30] J. Liu, Y. Chen, E. Atkinson, Y. Feng, and R. Bodik, “Conflict-driven synthesis for layout engines,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 132:1–132:22, 2023.
- [31] A. Zeller, R. Gopinath, M. Böhme, G. Fraser, and C. Holler, “Reducing failure-inducing inputs,” *The Fuzzing Book*, accessed June 2, 2026. [Online]. Available: <https://www.fuzzingbook.org/html/Reducer.html>
- [32] G. Mishergchi and Z. Su, “HDD: Hierarchical delta debugging,” in *Proceedings of the 28th International Conference on Software Engineering*, ser. ICSE ’06. ACM, 2006.
- [33] WebKit, “Test case reduction,” WebKit, accessed June 2, 2026. [Online]. Available: <https://webkit.org/test-case-reduction/>
- [34] StatCounter GlobalStats, “Browser market share worldwide,” StatCounter GlobalStats, May 2026, worldwide browser market share, May 2026; accessed June 24, 2026. [Online]. Available: <https://gs.statcounter.com/browser-market-share>